

Design RL Envs to Improve Frontier Agent's ML Engineering Capability

1. Context and Goal

We evaluate AI agents on their ability to do real ML engineering — such as PostTrainBench, where an agent receives a base model, a fixed compute budget, and an evaluation script, and must produce the best possible model.

Designing such tasks well is itself a hard research-engineering problem: the task must be solid (reproducible, deterministic scoring), valid (the metric measures the intended skill, with no leakage or shortcuts), and fair (achievable within budget, with meaningful headroom between the untuned base model and the reference baseline).

Your assignment is to design such tasks. We are evaluating your benchmark-design judgment as much as your ML engineering skill.

2. Your Assignment

Design 2-3 complete agent-evaluation RL envs in tracks of your choice (see the track menu below).

Each RL env tests a frontier agent's capability at solving a real-world ML engineering problem.

What each RL env should provide:

- **Clear instruction** (instruction.md): A clear instruction for the agent to understand the ML problem it is solving.
- **Verifier: The verifier should produce a single scalar score between 0 to 1**, that measures how well the agent did on the task.
 - **Rules preventing test-data contamination and model substitution..**
- **Oracle/Solution (optional)**: A solution that achieves a reasonably strong score.
- **Environment:**
 - **Compute budget for the task (defined in task.toml): one A100 or H100 GPU, OR CPU-only — 4 hours wall-clock**, all-inclusive (setup, training,

evaluation). CPU-only is legitimate (e.g., some scientific-ML tasks) but must still leave room for a real optimization loop.

- **1–2 pretrained base models** (from different families if you pick two — e.g., a ConvNeXt and a ViT, not two ResNets), small enough that a real improvement loop — not just one training run — fits in the budget.

Tracks (choose one: open-ended menu)

Pick any modality/domain where the core loop applies: pretrained model in, training decisions by the agent, held-out score out. Non-exhaustive menu:

Track	Spirit	Example directions (not prescriptive)
LLM post-training	Turn a small base LLM into a capable one on target skills	Math/GSM8K-style reasoning, code generation, function calling, domain QA
Vision adaptation	Adapt a pretrained backbone to new domains/conditions	Fine-grained classification, corruption robustness, low-label transfer, long-tail imbalance
Audio	Adapt a speech/audio model to a target condition	ASR on accented/noisy speech, keyword spotting, audio event detection, low-resource languages
Video	Adapt a video model under tight compute	Action recognition transfer, temporal localization, efficient frame-sampling regimes
Multimodal post-training	Post-train a small base VLM into a capable one	Document/chart QA, grounding, OCR-heavy reasoning, multimodal instruction following
Diffusion fine-tuning	Adapt a small generative model to a target constraint	Subject fidelity, prompt adherence (GenEval-style), domain style, efficient LoRA adaptation
Robotics / control	Improve a pretrained policy or world model in simulation	Behavior cloning refinement, sim manipulation suites, offline RL on fixed datasets
Scientific ML	Improve a scientific model on held-out systems	Molecular property prediction, interatomic potentials, PDE surrogates, protein property regression

3D / CAD	Adapt a 3D understanding or generation model	Point-cloud classification/segmentation transfer, mesh/CAD retrieval, single-view reconstruction
Other	Any domain fitting the core loop	Tabular foundation models, time-series forecasting, graph learning, RecSys... — propose it

3. Packaging requirement: Harbor task format

The task must be packaged in Harbor format (harborframework.com) — the standard agent-task layout used by Terminal-Bench-style harnesses:

```
<org>/<task-name>/
├── task.toml      # metadata; [environment] resources (GPU, memory);
│                  # [agent] timeout_sec = your 4h budget
├── instruction.md  # agent-facing prompt (goal, rules, budget — no hints)
├── environment/   # Dockerfile defining the agent workspace
├── solution/      # optional solve.sh — your probe fine-tune
└── tests/         # test.sh → reward to /logs/verifier/reward.json
```

Use separate-verifier mode (a dedicated verifier container) so held-out test data and grading logic never enter the agent's container. The verifier writes `/logs/verifier/reward.json` (multi-metric) with `reward.txt` as fallback; encodes your aggregate scalar as the primary reward and per-eval-set scores as additional metrics. harbor init scaffolds the layout.

4. Rules You Must Design (and Enforce)

Anti-contamination

- Held-out test sets are invisible to the agent — not in the workspace, not downloadable by name. Final scoring runs on splits the agent never touches. Explain your split scheme and how you verified the test data isn't publicly available in a form the agent could fetch (or what you did to make it so: private splits, held-back perturbations, fresh generations).

- The agent may build or download its own training/validation data — that freedom is the point — so your contamination rules must be about the test side, not about restricting the agent's data choices.

Anti-substitution

The submitted checkpoint must be derived from the provided base model, not a swapped-in stronger model. Specify your enforcement mechanism, e.g.: architecture/config hash must match the provided base; weight-provenance spot checks (correlation between submitted and base weights outside the layers a legitimate fine-tune would move most); required training logs; automatic rejection if the submitted model is bit-identical to any public checkpoint other than the provided base.

Scoring integrity

- **Scoring is fully automated:** one command → final score, no human judgment.
- **Objective, ground-truth metrics only.** The reward must be computed from objective quantities against held-out ground truth — accuracy, loss/perplexity, WER, mAP, MAE, exact-match, unit-test pass rate, and the like. Rubric-based grading and free-form LLM/VLM-judge scoring are not acceptable as the reward: they are noisy, driftable, and promptable — an agent optimizing against a judge learns to please the judge, not to improve the model.
- **No paid external model APIs in the solution path** (mirrors PostTrainBench v1.1 rules).
- **Everything pinned:** dataset versions/checksums, model revisions, package versions, seeds, container/environment definition.

Reward / verifier reliability (required)

The verifier is the single source of truth for the score, so its reliability is a first-class requirement, not an afterthought:

- **Objective by construction.** The reward is a mechanical computation against held-out ground truth (per Scoring integrity above) — no rubric, no judge, no human in the loop.
- **Never crashes into ambiguity.** Any agent outcome — missing checkpoint, malformed or truncated files, wrong dtype/shape, a model that errors at inference, adversarially weird outputs — must yield a valid reward (typically 0 or the base-model floor), never a verifier exception or an absent reward.json. Assume the agent's output is arbitrary and potentially adversarial.

- **Isolated from the agent.** Run in separate-verifier mode; the agent must not be able to read, import, or overwrite grading code, judge prompts, or hold-out data from its container. Treat anything the agent can write as untrusted input to the verifier.
- **Deterministic (or variance-bounded).** Same checkpoint in $\rightarrow$ same reward out; where full determinism is impossible, report measured reward variance and show it is well below the base $\rightarrow$ reference gap.
- **Calibration-tested before submission.** Run the verifier on at least: the untouched base model, the reference counterpart, your probe fine-tune, a deliberately broken checkpoint, and (if feasible) a deliberate reward-hacking attempt. Report the reward each produced and confirm the ordering matches expectation. A verifier that has only ever seen "good" inputs is untested.

Verifier reliability is graded under Solidity (crash-safety, determinism, calibration) and Validity (hackability) in the rubric — a verifier the agent can crash or game caps both.

5. Validation and Difficulty Measurement

Execute one live agent run against your task:

- Agent model: gpt-5.6-sol · Agent harness: Codex CLI · $k = 1$ (single attempt), launched via harbor run against your task directory as submitted.
- The agent receives exactly your instruction.md, your environment, and your compute budget ([agent].timeout_sec).
- Your verifier (tests/test.sh) produces /logs/verifier/reward.json; we compute normalized recovery against your reported base and reference anchors, and your anti-substitution check runs as part of verification.

A note on $k = 1$: a single run is noisy, and we treat it that way — it is a smoke test of your task's solidity and fairness, not a leaderboard.